

CS 1675 Final Report

Hate Obliterator 10000

Austin Gohn, Diana Lysova, Ryder Pham

Abstract

This project studies hate speech detection on short social media text. Given a tweet, the task is to classify it as hate speech, offensive language, or neither. We compare several classical machine learning baselines using n -gram features against a fine-tuned DistilBERT model. Because the dataset is class-imbalanced and hate speech is the rarest class, we evaluate models using class-specific precision, recall, and F1 rather than accuracy alone. Our results show that DistilBERT achieves the strongest macro-F1 score and substantially improves recall on the hate-speech class, although this comes with lower hate-speech precision than some simpler baselines. We analyze this tradeoff through variance across random seeds and qualitative failure cases.

Code —

<https://github.com/dianaelysova/HateObliterator10000>

Dataset — <https://www.kaggle.com/datasets/mrmorj/hate-speech-and-offensive-language-dataset>

Introduction

Online platforms contain large volumes of user-generated text, making manual moderation difficult to scale. As social media platforms have become a primary means of communication throughout the world, hate speech and harmful content have also proliferated. Because this kind of content can have many negative consequences, especially for younger users, there needs to be a way to identify hate speech online. The distinction between hate speech and offensive language is central to the project. A model that simply detects profanity may perform well on offensive language detection, but fail to identify whether the post expresses hate or targets a protected group. Our project, the Hate Obliterator 10000, aims to utilize machine learning methods to classify a string of text as hate speech or neutral language, allowing inappropriate content to be filtered out. Our goal is not only to build a high-accuracy text classifier, but to evaluate the performance of different models when they perform the task. We compare traditional machine learning models using n -gram features with a fine tuned DistilBERT model. The traditional models provide interpretable baselines, while DistilBERT

provides contextual representations that may better capture word meaning, phrase structure, and implicit context. Our experiments show that DistilBERT performs best overall by macro-F1 and achieves the highest recall on the hate-speech class, but simpler models sometimes have higher precision for hate speech. However, DistilBERT has the best balance among all classes and detects the majority of hate-speech, which suggests it is the best model.

Dataset Description

The dataset used contains a list of roughly 25,000 tweets sourced from X (formerly Twitter). Each tweet is labeled with the number of people who labeled the tweet as hate speech, the number of people who labeled the tweet as offensive, the number of people who labeled the tweet as neither, the total number of people who labeled the tweet, and the decided classification of the tweet as hate, offensive, or neither. Class 0 contains the instances of hate speech, class 1 the instances of offensive speech, and class 2 the instances of neither. There is also a final label derived from the aggregated decision. It is noteworthy that the dataset is imbalanced—class 0 contains 1430 tweets, while classes 1 and 2 contain 19190 and 4163 respectively. The implications of this will be discussed during comparison of DistilBERT with the baseline models.

Project Description

As discussed in the project proposal, this project aims to classify a string of text as inappropriate or not, allowing inappropriate content to be filtered out. This will be done using the previously described dataset. Mathematically, we can define the problem as follows:

- Feature space $\mathcal{X} \subseteq \mathbb{R}^d$, where d is the TF-IDF vocabulary size and we are considering n -gram features.
- Label space

$$\mathcal{Y} = \{\text{hateful, offensive, neither}\}.$$

In the project proposal we originally were going to simplify the classification to be either hateful/offensive or neither, but decided to instead stay consistent with the dataset and have the classifications be hateful, offensive, or neither.

The training dataset can be defined as

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n,$$

where

- $x^{(i)}$ is the i -th text sample,
- $y^{(i)} = 0$ if $x^{(i)}$ is deemed hateful, 1 if offensive, and 2 if neither.

Therefore, we are (ideally) aiming to find the target function $C^* : \mathcal{X} \rightarrow \mathcal{Y}$ that can correctly predict whether a text sample $x \in \mathcal{X}$ is hate speech. Thus, our problem becomes finding a classifier $h : \mathcal{X} \rightarrow \mathcal{Y}$ that best approximates C^* . In order to do this, we need to minimize the loss function

$$\ell(y, \hat{y}) = \mathbb{1}(y \neq \hat{y}).$$

Related Work

Prior work on hate speech detection can be organized along two dimensions that are directly relevant to this project: the application dimension, which concerns the existing harmful-language detection problem being solved, and the technical dimension, which concerns what type of model is used to solve it. Our work is closest to prior X (Twitter) hate speech detection research in its application setting, but differs in its label structure and evaluation priorities. Our work combines classical machine learning baselines with a tuned DistilBERT model to compare interpretable sparse-feature methods against a contextual transformer based approach.

Application Dimension: Hate Speech Detection on Twitter

Khanday et al. study hate speech detection on X (Twitter) during the COVID-19 period using machine learning and ensemble learning methods (Khanday et al. 2022). This work is relevant to our project because it treats social media moderation as a supervised text-classification problem, using Twitter data and automatic feature extraction to identify harmful content. Like our project, the study’s motivation is the difficulty in manually moderating large volumes of content, and by the fact that social media use can foster harmful language and hate speech. Their work uses traditional text representations, such as TF-IDF and Bag Of Words features, which are similar to the sparse n-gram representations used in our logistic regression, AdaBoost, and Random Forest baselines. However, our task differs from Khanday et al.’s in a significant way: instead of focusing on detecting hate speech in the context of COVID-19, our model is trained on a broader set of tweets labeled as hate speech, offensive language, or neither. This is an important difference because COVID specific datasets may contain topical clues that are associated with hate speech during that specific event, but our dataset requires the model to distinguish hate speech from offensive language in a wider range of contexts. Further, our three classes make the problem more difficult than a binary approach, as used in Khanday et al.’s work: a tweet can be profane without necessarily being hate speech. Their model, which sorts tweets containing explicit words into the ”Hate” class, may perform well on offensive language detection while still failing to identify hate speech. Khanday et al. report strong results for classical and ensemble methods, especially decision trees and stochastic gradient boosting (Khanday et al. 2022). These results provide motivation for includ-

ing ensemble baselines in our own work. However, our results show that ensemble methods are not automatically effective for every hate speech setting. In our work, AdaBoost and Random Forests performed poorly on the hate speech class, with AdaBoost producing an F_1 score of 0 for hate speech and Random Forests producing very low hate speech recall (see tables 1 and 3). This difference indicates that the success of classical ensemble methods depends heavily on the dataset, label structure, feature representation, and class balance. Since our dataset is extremely unbalanced, with hate speech being the minority class, models can achieve reasonable overall performance by predicting the majority offensive class, while still failing to detect hate speech. Due to this, our work emphasizes balance between precision, recall, and F_1 rather than one measure alone. Khanday et al.’s work shows that classical and ensemble methods can be effective for certain specific hate speech detection tasks, but our broader setting requires more holistic analysis.

Technical Dimension: DistilBERT

The second relevant work is Sanh et al.’s DistilBERT model (Sanh et al. 2020). DistilBERT is a compressed version of BERT produced through knowledge distillation. The goal of the model is to preserve much of BERT’s contextual language understanding while reducing model size and computational cost. This is important for our project because hate speech detection depends on context. Sparse features, such as n-grams, can capture words and short phrases, but they do not fully model how word meaning changes across contexts. This is especially important in our case, since the difference between hate speech and offensive language depends on whether the text targets a protected group, word combinations, and whether the sentence expresses hostility rather than general profanity. DistilBERT is a good technical choice for our goal because it strikes a balance between capability and practicality. A full BERT model would provide contextual embeddings, but it is much more computationally expensive to train and fine tune. DistilBERT is smaller and faster, which gives it limited compute but retains the transformer based contextual representations. Using DistilBERT gives an advantage over classical baselines, since the model processes the sequence of words as context dependent language instead of as weighted counts of tokens or n-grams. When evaluating whether a tweet is hateful, offensive, or neither, DistilBERT can use information from the entire tweet. Our results support the initial motivation for using DistilBERT, which achieves the strongest macro level performance among the tested models. Across five random seeds, DistilBERT achieves a mean macro F_1 of 0.7659 and a mean macro recall of 0.8144. It also reaches a hate speech recall of 0.6014, which is much higher than the recall of baseline models (see table 5). This shows that DistilBERT is much better at finding actual instances of hate speech rather than just performing well on the majority class in the dataset. Taken together, these two prior works explain the structure of and motivation for our project. Khanday et al. show that ensemble methods achieve good results in binary ”Hate” and ”Normal” classes, and motivate us to explore both these classical models as well as a more powerful model in Dis-

tilBERT for broader contexts on social media and an extra class. Sanh et al. motivate our use of DistilBERT as a more computationally efficient contextual model.

Logistic Regression: Feature Engineering and Training

During the past several weeks we have been working on developing the logistic regression model (i.e., a 1-layer neural network) as well as training and testing it on the discussed dataset. All of our code is currently in one Python file named `nlp_train.py`. The algorithm starts by splitting the dataset into data for training and testing, using 80% of the data for training and 20% of the data for testing.

After splitting the dataset, we transformed the raw text into features suitable for training and testing. This involved two different methods. TF-IDF vectorization was used to identify key words that signify text as hateful or offensive, and n -gram extraction was used to capture the context of phrases. In other words, TF-IDF was used to find words that signify hate speech, while n -grams were used to see if the words are hate speech in the context of the tweet. We also included several custom features: average word length, number of ellipses, number of new lines, number of commas, number of periods, number of question marks, number of exclamation points, number of semicolons, number of colons, number of parentheses, comma rate, capital rate, and average sentence length. All of the features were then standardized using scikit-learn's `StandardScaler` to ensure that all features had the same standard deviation during classification.

To determine the most effective approach for logistic regression classification, we trained three iterations of a logistic regression model. Our first baseline model (*vanilla unigram model*) was trained using only the TF-IDF unigrams. The second model (*combined unigram model*) was trained using the TF-IDF unigrams as well as our custom features. The final logistic regression model (*combined n -gram model*) used unigrams, bigrams, and trigrams, as well as our custom features.

Logistic Regression: Testing and Current Results

We decided to evaluate the model using three metrics: precision, recall, and F_1 -score. Precision measures the fraction of true positives over all retrieved instances for each category, and recall measures the fraction of true positives over all relevant instances for each category. That is,

$$\text{precision} = \frac{tp}{tp + fp}, \quad \text{recall} = \frac{tp}{tp + fn},$$

where tp represents the count of true positives, fp the count of false positives, and fn the count of false negatives. The F_1 -score measures the overall performance of the model by calculating the harmonic mean of precision and recall; that is,

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Note that since we have many more examples in the test dataset for category 1, accuracy is not a good metric for evaluating performance.

As shown in Tables 1–3, the precision, recall, and F_1 -score of all three models was very good for offensive speech (1) and neither (2). However, all three models are lacking when it comes to hate speech. Specifically, recall for the Vanilla, Combined Unigram, and Combined n -gram models for category 0 are 0.18, 0.23, and 0.20 respectively. This means that the Combined Unigram Model is performing the best in terms of recall, but our models fail to identify tweets that are actually hateful, as opposed to simply being offensive. The F_1 -score is also very low for all three models, with the Combined Unigram model performing the best according to this metric. As a whole, the Combined Unigram Model is currently performing the best overall. More exploration and experimentation will need to be done to determine how to raise the recall and F_1 -score of the models for category 0.

AdaBoost

The next baseline model we used was AdaBoost to see how well an ensemble method would perform on classifying the data. The AdaBoost model was tested using both the Combined Unigram features as well as the Combined Trigram features. Looking at table 1, the F_1 scores for both AdaBoost models underperformed all models for all tweets regardless of category.

Now looking at Table 2, the Precision for both AdaBoost models underperformed other models for tweets that were neither hate speech nor offensive. However, it outperformed all models when it came to classifying offensive tweets.

According to table 3, the Recall drastically outperformed when it came to classifying tweets that were neither hate speech nor offensive but underperformed when it came to classifying hate speech and offensive tweets.

It should be mentioned that running both models of AdaBoost resulted in F_1 scores, Precision, and Recall values of 0 for hate speech. This means that AdaBoost failed to predict any tweets as hate speech and predicted nearly all tweets as being neither hate speech nor offensive. The reason why this occurred is because of the way AdaBoost works. AdaBoost was trained on decision stumps which are limited to splitting based on a single feature. Since hate speech is rare in this dataset due to it being unbalanced, a single word does not provide enough context to accurately classify the tweet. Thus, the models defaulted to Neither to reduce the overall error.

Random Forests

Another ensemble model that we chose to use as a baseline is Random Forests. Like AdaBoost we trained Random Forests on both combined Unigram features as well as Combined Trigram features. Looking at table 1, in terms of the F_1 score, Random Forests performed well for Offensive language and not as well compared to other models for Neither. For classifying Hate Speech, the F_1 scores were very close to 0 for Random Forests.

Looking at table 2, Random Forests performed similar to other models in all categories in terms of precision. Random Forests in particular over performed other models when it came to classifying tweets as Neither. Looking at table 3, Random Forests had a very low recall close to 0 for hate speech but had a high recall for offensive tweets.

Similar to AdaBoost, Random Forests often defaulted to labeling a tweet as offensive and very rarely labeled a tweet as Hate Speech. This is because Random Forests builds decision trees by only evaluating a random subset of features at each split. Because there are so many features, many are irrelevant to a specific tweet. The model is usually looking at unhelpful words to classify a tweet as Hate Speech. To help minimize the error, Random Forests usually defaults to labeling tweets as Offensive

Discussion of the Models

To compare DistilBERT with the baseline models, we evaluated their scores on accuracy, precision, recall, and F1. Combined n-grams had the highest accuracy, with a 0.9032, however, this result is misleading on its own, especially when the dataset is imbalanced. In our dataset, the majority of data points are in the second class, offensive language. Unigrams are the most precise model, with a precision of 0.8161, but this comes at the cost of F1 and recall, which are only 0.65. When considering F1 and recall, DistilBERT outperforms all baselines by a significant amount, with 0.7476 and 0.8150 respectively. Although it does not outperform the baseline models in all categories, DistilBERT has the best balance amongst all classes. It is also worth comparing DistilBERT's recall scores across classes with the baseline models. DistilBERT outperforms all baselines in the macro average recall, as well as the recall for class 0 in the dataset by a significant amount- it's scores are 0.8150 and 0.6224 respectively. However, it does not achieve the highest recall for the remaining classes, class 1 and 2. Random Forest achieves a near perfect recall for class 1 (0.9930), and AdaBoost performs similarly for class 2 (0.9988). Despite this, DistilBERT remains the best performing model, as the dataset is highly imbalanced. Baseline models are able to achieve high accuracy, precision and recall in classes 1 and 2 by predicting in the majority class (offensive language), which contains 19190 instances compared to the much smaller 1430 in the hate speech class. However, they do not achieve as high a score as DistilBERT on the minority class, which is most important for the purposes of this project. Overall, DistilBERT is much more balanced across classes: its scores for accuracy, precision, and recall on classes 1 and 2 are still above 0.7, which indicates good performance, even if not the highest score among all models. DistilBERT's superiority to the baseline models can be shown clearly, however, in its recall on class 0: it achieves a score of 0.6224, while other models give 0.181, 0.231, 0.206, 0.052, 0.014, 0.0. (See Table 3).

A Deeper Look at the DistilBERT Model

To assess the stability of the fine-tuned DistilBERT model, we trained it five times using identical hyperparameters and data splits but different random seeds (0–4), varying

only the model weight initialization and training stochasticity. The data split was held fixed across all runs using `random_state=42`, ensuring that any observed variation reflects training randomness rather than differences in data partitioning.

Referring to Table 4, across the five runs, the model exhibited very low variance on aggregate metrics. Accuracy had a mean of 0.8935 with a standard deviation of 0.0052, and macro F1 averaged 0.7659 with a standard deviation of just 0.0042. This stability is expected for fine-tuning a large pre-trained model: the vast majority of DistilBERT's parameters are loaded from a pre-trained checkpoint, with only the classification head randomly initialized. The random seed therefore has minimal influence on the final model compared to the strong pre-trained prior.

However, variance was not uniform across classes. The offensive and neither classes were highly stable, achieving mean F1 scores of 0.9351 and 0.8981 respectively with negligible run-to-run fluctuation. The hate_speech class was considerably less stable, with precision ranging from 0.36 to 0.41 and recall ranging from 0.55 to 0.65 across runs. This instability stems from the severe class imbalance — hate speech constitutes only 6% of the dataset (143 of 2,479 test examples) — meaning that even small shifts in the decision boundary can meaningfully change per-class recall. Macro recall, which is sensitive to the minority class, had the highest standard deviation of the four reported metrics (0.0100).

These results suggest that a single training run is sufficient to obtain a representative estimate of model performance on the majority classes, but conclusions about hate speech detection specifically should be drawn cautiously, as that class remains sensitive to training randomness even under class-weighted loss.

Future Work

Since the progress report, we have implemented AdaBoost, Random Forests, and DistilBERT. We have also shown how DistilBERT compares to the baseline models and have discussed why DistilBERT outperforms other models.

In the future, we could explore how our baseline models perform on a balanced dataset and then compare the baselines to DistilBERT. This would allow us to see how much better DistilBERT is on a balanced dataset. We could also explore how to tune the baseline models (specifically AdaBoost and Random Forests) to handle unbalanced datasets. In addition, we could look into what other pre trained models exists and how they perform compared to DistilBERT.

Tooling and Workflow Transparency

We used Claude Code (Anthropic's CLI coding agent) primarily for code generation of the DistilBERT fine-tuning experiments and variance analysis, albeit with much prodding and re-prompting. For instance, in generating the code for the variance analysis, a bug was introduced in which the models were being re-trained instead of using their saved checkpoints. We also had to iteratively prompt Claude Code to add more features as our needs changed.

References

- Davidson, T.; Warmusley, D.; Macy, M.; and Weber, I. 2017. Hate Speech and Offensive Language Dataset.
- Khanday, A. M. U. D.; Rabani, S. T.; Khan, Q. R.; and Malik, S. H. 2022. Detecting twitter hate speech in COVID-19 era using machine learning and ensemble learning techniques. *International Journal of Information Management Data Insights*, 2(2): 100120.
- Sanh, V.; Debut, L.; Chaumond, J.; and Wolf, T. 2020. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.

Results Tables

Table 1: F_1 -scores by model and class.

Model	Hate Speech	Offensive	Neither
Vanilla Unigram	0.287	0.940	0.838
Combined Unigram	0.340	0.942	0.843
Combined Trigram	0.305	0.943	0.856
AdaBoost Combined Unigram	0.000	0.571	0.397
AdaBoost Combined Trigram	0.000	0.571	0.397
Random Forest Combined Unigram	0.096	0.919	0.696
Random Forest Combined Trigram	0.027	0.896	0.445
DistilBERT	0.476	0.934	0.898

Table 2: Precision by model and class.

Model	Hate Speech	Offensive	Neither
Vanilla Unigram	0.675	0.911	0.861
Combined Unigram	0.647	0.918	0.850
Combined Trigram	0.584	0.925	0.840
AdaBoost Combined Unigram	0.000	0.974	0.247
AdaBoost Combined Trigram	0.000	0.974	0.247
Random Forest Combined Unigram	0.600	0.860	0.898
Random Forest Combined Trigram	0.571	0.816	0.883
DistilBERT	0.320	0.970	0.856

Table 3: Recall by model and class.

Model	Hate Speech	Offensive	Neither
Vanilla Unigram	0.181	0.971	0.816
Combined Unigram	0.231	0.966	0.836
Combined Trigram	0.206	0.961	0.874
AdaBoost Combined Unigram	0.000	0.404	0.999
AdaBoost Combined Trigram	0.000	0.404	0.999
Random Forest Combined Unigram	0.052	0.987	0.568
Random Forest Combined Trigram	0.014	0.993	0.298
DistilBERT	0.622	0.880	0.942

Table 4: DistilBERT Test Set Performance Across 5 Random Seeds

Run (Seed)	Accuracy	Macro F1	Macro Precision	Macro Recall
1 (seed=0)	0.8996	0.7685	0.7401	0.8068
2 (seed=1)	0.8887	0.7645	0.7330	0.8254
3 (seed=2)	0.8975	0.7728	0.7453	0.8172
4 (seed=3)	0.8862	0.7627	0.7307	0.8235
5 (seed=4)	0.8955	0.7612	0.7347	0.7991
Mean	0.8935	0.7659	0.7367	0.8144
Std	0.0052	0.0042	0.0053	0.0100

Table 5: DistilBERT Per-Class Test Set Performance (Mean across 5 Seeds)

Class	Precision	Recall	F1	Support
Hate Speech	0.3806	0.6014	0.4646	143
Offensive	0.9661	0.9061	0.9351	1919
Neither	0.8635	0.9357	0.8981	417
Macro Avg	0.7367	0.8144	0.7659	2479